Dynamic Programming

Two worked problems: rod cutting and edit distance

AIL7002 · Data Structures and Algorithms (refresher)

IIT Delhi

Today: dynamic programming by example

Dynamic programming = recursion + remembering.

- Write the optimum as a recurrence over *smaller* instances of the same problem.
- The naive recursion is exponential because it solves the same subproblems again and again.
- There are only polynomially many distinct subproblems: solve each once, store it in a table.

Bellman, 1950s: "programming" = filling in a table (planning), not writing code.

Problem	**Table**
Rod cutting	1-D: $r_0, \ldots, r_n$
Edit distance	2-D: $D[0..m][0..n]$

For each: *subproblem* $\rightarrow$ *recurrence* $\rightarrow$ *order* $\rightarrow$ *table* $\rightarrow$ *solution*.

Rod cutting

The rod-cutting problem

Input: a rod of length n and prices p_i for a piece of length $i = 1, \ldots, n$.

Goal: cut it into integer pieces to maximise the revenue $r_n = \sum p_{\text{piece}}$.

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

The rod-cutting problem

Input: a rod of length n and prices p_i for a piece of length $i = 1, \ldots, n$.

Goal: cut it into integer pieces to maximise the revenue $r_n = \sum p_{\text{piece}}$.

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

A cut can be made or not at each of the $n - 1$ marks: 2^{n-1} ways to cut.

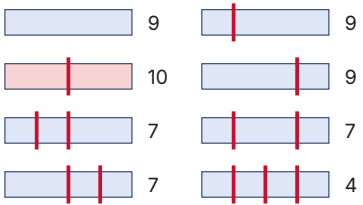
The rod-cutting problem

Input: a rod of length n and prices p_i for a piece of length $i = 1, \ldots, n$.

Goal: cut it into integer pieces to maximise the revenue $r_n = \sum p_{\text{piece}}$.

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

A cut can be made or not at each of the $n - 1$ marks: 2^{n-1} ways to cut. For $n = 4$ the best is $2 + 2$: revenue $5 + 5 = 10 > p_4 = 9$.



All $2^3 = 8$ ways to cut a rod of length 4 (red bars = cuts, number = revenue).

Optimal substructure: think about the first piece

In an optimal cut of a rod of length n, the leftmost piece has *some* length $i \in \{1, \ldots, n\}$.

Optimal substructure: think about the first piece

In an optimal cut of a rod of length n, the leftmost piece has *some* length $i \in \{1, \ldots, n\}$.

Claim. The remaining length $n - i$ is then cut optimally too.

Cut-and-paste: if the remainder earned less than r_{n-i}, replace its cut by an optimal one $\Rightarrow$ more revenue for n, a contradiction.

Optimal substructure: think about the first piece

In an optimal cut of a rod of length n, the leftmost piece has *some* length $i \in \{1, \ldots, n\}$.

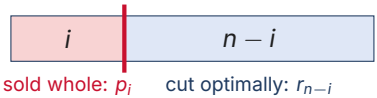
Claim. The remaining length $n - i$ is then cut optimally too.

Cut-and-paste: if the remainder earned less than r_{n-i}, replace its cut by an optimal one $\Rightarrow$ more revenue for n, a contradiction.

We do not know i — so try all n choices:

Recurrence

$$r_0 = 0, \qquad r_n = \max_{1 \le i \le n} \left(p_i + r_{n-i} \right) \quad (n \ge 1).$$



- One subproblem per length: $r_0, r_1, \ldots, r_n$.
- $i = n$ means "no cut": $p_n + r_0$.
- Each r_j depends only on smaller r's.

Direct recursion is exponential

Cut-Rod(p, n)

if $n = 0$ **return** 0
$q \leftarrow -\infty$
for $i = 1$ **to** n
  $q \leftarrow \max(q,\ p_i + \text{Cut-Rod}(p, n-i))$
return q

Let $T(n)$ = number of calls made by Cut-Rod(p, n):

$$T(0) = 1, \qquad T(n) = 1 + \sum_{j=0}^{n-1} T(j).$$

Direct recursion is exponential

Cut-Rod(p, n)

if $n = 0$ **return** 0
$q \leftarrow -\infty$
for $i = 1$ **to** n
  $q \leftarrow \max(q,\ p_i + \text{Cut-Rod}(p, n-i))$
return q

Let $T(n)$ = number of calls made by Cut-Rod(p, n):

$$T(0) = 1, \qquad T(n) = 1 + \sum_{j=0}^{n-1} T(j).$$

Then $T(n) = 2^n$ (induction: $1 + (2^0 + \cdots + 2^{n-1}) = 2^n$).

Direct recursion is exponential

Cut-Rod(p, n)

if $n = 0$ **return** 0
$q \leftarrow -\infty$
for $i = 1$ **to** n
  $q \leftarrow \max(q,\ p_i + \text{Cut-Rod}(p, n-i))$
return q

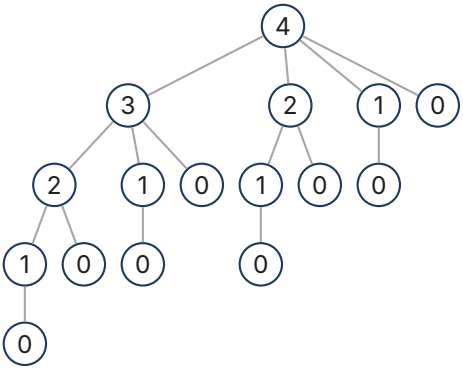
Let $T(n)$ = number of calls made by Cut-Rod(p, n):

$$T(0) = 1, \qquad T(n) = 1 + \sum_{j=0}^{n-1} T(j).$$

Then $T(n) = 2^n$ (induction: $1 + (2^0 + \cdots + 2^{n-1}) = 2^n$).

Correct, but already slow for $n \approx 40$. Where does the time go?

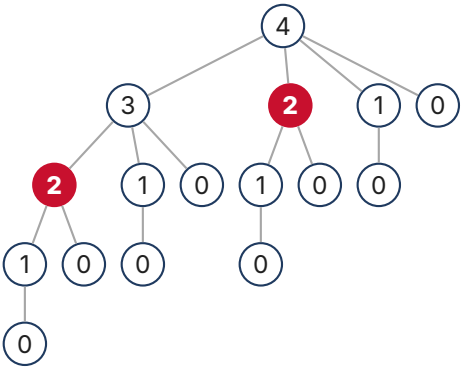
The recursion tree for $n = 4$: the same work, over and over



Cut-Rod$(p, 4)$: a node labelled k is a call on a rod of length k; its children try every first piece $i = 1, \ldots, k$.

16 calls $= 2^4$ for $n = 4$.

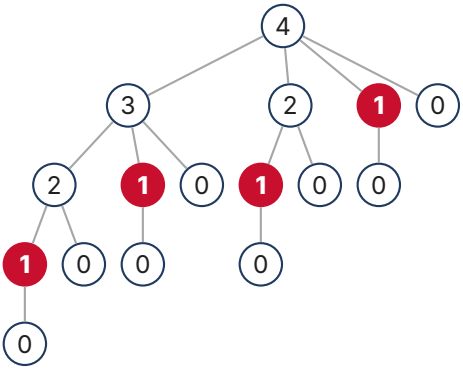
The recursion tree for $n = 4$: the same work, over and over



Subproblem r_2 is solved 2 times.

In general r_k is recomputed 2^{n-k-1} times.

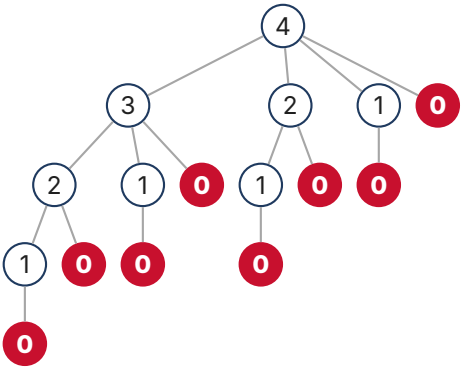
The recursion tree for $n = 4$: the same work, over and over



Subproblem r_1 is solved 4 times.

In general r_k is recomputed 2^{n-k-1} times.

The recursion tree for $n = 4$: the same work, over and over



r_0 is reached 8 times – once per leaf.

Yet there are only $n + 1 = 5$ *distinct* sub-problems $r_0, \ldots, r_4$.

Store each answer once.

Remember the answers: two equivalent ways

Top-down (memoisation): same recursion; before computing r_n, look it up; after, store it. Added – compiled and tested:

```
// Top-down: r[k] = -1 means "not solved yet".
int memo_cut(const vector<int>& p, int n,
             vector<int>& r) {
    if (r[n] >= 0) return r[n];    // look up
    int best = (n == 0) ? 0 : INT_MIN;
    for (int i = 1; i <= n; ++i)
        best = max(best, p[i] + memo_cut(p, n - i, r));
    return r[n] = best;            // store
}
```

Call with `r` filled with -1 (“unknown”).

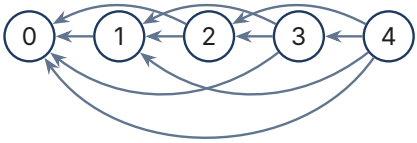
Remember the answers: two equivalent ways

Top-down (memoisation): same recursion; before computing r_n, look it up; after, store it. Added – compiled and tested:

```
// Top-down: r[k] = -1 means "not solved yet".
int memo_cut(const vector<int>& p, int n,
             vector<int>& r) {
    if (r[n] >= 0) return r[n];    // look up
    int best = (n == 0) ? 0 : INT_MIN;
    for (int i = 1; i <= n; ++i)
        best = max(best, p[i] + memo_cut(p, n - i, r));
    return r[n] = best;            // store
}
```

Call with `r` filled with -1 ("unknown").

Bottom-up: fill $r_0, r_1, \ldots, r_n$ in increasing order — every look-up is already in the table.



Subproblem graph: an edge $j \to j - i$ for each choice i. A DAG with $n + 1$ nodes.

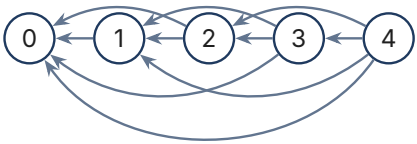
Remember the answers: two equivalent ways

Top-down (memoisation): same recursion; before computing r_n, look it up; after, store it. Added – compiled and tested:

```
// Top-down: r[k] = -1 means "not solved yet".
int memo_cut(const vector<int>& p, int n,
             vector<int>& r) {
    if (r[n] >= 0) return r[n];    // look up
    int best = (n == 0) ? 0 : INT_MIN;
    for (int i = 1; i <= n; ++i)
        best = max(best, p[i] + memo_cut(p, n - i, r));
    return r[n] = best;            // store
}
```

Call with `r` filled with -1 ("unknown").

Bottom-up: fill $r_0, r_1, \ldots, r_n$ in increasing order — every look-up is already in the table.



Subproblem graph: an edge $j \to j - i$ for each choice i. A DAG with $n + 1$ nodes.

Running time

r_j costs j steps $\Rightarrow \sum_{j=1}^{n} j = \Theta(n^2)$, space $\Theta(n)$.
From 2^n to n^2.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0								
first piece s_j	–								

Base case: a rod of length 0 earns nothing, $r_0 = 0$. Now fill $r_1, r_2, \ldots$ left to right: every r_j needs only *smaller* entries.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0	1							
first piece s_j	–	1							

$p_1 + r_0 = 1 + 0 = 1$

$r_1 = \max_{1 \le i \le 1}(p_i + r_{1-i}) = 1$, achieved by a first piece of length $s_1 = 1$. Only 1 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	**5**	8	9	10	17	17	20

rod j	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
r_j	0	1	**5**						
first piece s_j	–	1	2						

$p_1 + r_1 = 1 + 1 = 2$

$\mathbf{p_2 + r_0 = 5 + 0 = 5}$

$r_2 = \max_{1 \le i \le 2}(p_i + r_{2-i}) = 5$, achieved by a first piece of length $s_2 = 2$. Only 2 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	5	**8**	9	10	17	17	20

rod j	**0**	1	2	3	4	5	6	7	8
r_j	0	1	5	**8**					
first piece s_j	–	1	2	3					

$p_1 + r_2 = 1 + 5 = 6$
$p_2 + r_1 = 5 + 1 = 6$
$\mathbf{p_3 + r_0 = 8 + 0 = 8}$

$r_3 = \max_{1 \le i \le 3}(p_i + r_{3-i}) = 8$, achieved by a first piece of length $s_3 = 3$. Only 3 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	**5**	8	9	10	17	17	20

rod j	**0**	1	2	3	**4**	5	6	7	8
r_j	0	1	5	8	**10**				
first piece s_j	–	1	2	3	2				

$p_1 + r_3 = 1 + 8 = 9$
$\mathbf{p_2 + r_2 = 5 + 5 = 10}$
$p_3 + r_1 = 8 + 1 = 9$
$p_4 + r_0 = 9 + 0 = 9$

$r_4 = \max_i(p_i + r_{4-i}) = 10$, first piece 2, then r_2 says: piece 2.
Two halves beat selling whole ($p_4 = 9$).

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	**5**	8	9	10	17	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0	1	5	8	10	**13**			
first piece s_j	–	1	2	3	2	2			

$p_1 + r_4 = 1 + 10 = 11$
$\mathbf{p_2 + r_3 = 5 + 8 = 13}$
$p_3 + r_2 = 8 + 5 = 13$
$p_4 + r_1 = 9 + 1 = 10$
$p_5 + r_0 = 10 + 0 = 10$

$r_5 = \max_{1 \le i \le 5}(p_i + r_{5-i}) = 13$, achieved by a first piece of length $s_5 = 2$. Only 5 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	**17**	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0	1	5	8	10	13	**17**		
first piece s_j	–	1	2	3	2	2	6		

$p_1 + r_5 = 1 + 13 = 14$
$p_2 + r_4 = 5 + 10 = 15$
$p_3 + r_3 = 8 + 8 = 16$
$p_4 + r_2 = 9 + 5 = 14$
$p_5 + r_1 = 10 + 1 = 11$
$\mathbf{p_6 + r_0 = 17 + 0 = 17}$

$r_6 = \max_{1 \le i \le 6}(p_i + r_{6-i}) = 17$, achieved by a first piece of length $s_6 = 6$. Only 6 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	5	8	9	10	17	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0	1	5	8	10	13	17	18	
first piece s_j	–	1	2	3	2	2	6	1	

$\mathbf{p_1 + r_6 = 1 + 17 = 18}$
$p_2 + r_5 = 5 + 13 = 18$
$p_3 + r_4 = 8 + 10 = 18$
$p_4 + r_3 = 9 + 8 = 17$
$p_5 + r_2 = 10 + 5 = 15$
$p_6 + r_1 = 17 + 1 = 18$
$p_7 + r_0 = 17 + 0 = 17$

$r_7 = \max_{1 \le i \le 7}(p_i + r_{7-i}) = 18$, achieved by a first piece of length $s_7 = 1$. Only 7 look-ups into the table.

Bottom-up, step by step

length i	1	2	3	4	5	6	7	8
price p_i	1	**5**	8	9	10	17	17	20

rod j	0	1	2	3	4	5	6	7	8
r_j	0	1	5	8	10	13	17	18	**22**
first piece s_j	–	1	2	3	2	2	6	1	2

$$p_1 + r_7 = 1 + 18 = 19$$
$$\mathbf{p_2 + r_6 = 5 + 17 = 22}$$
$$p_3 + r_5 = 8 + 13 = 21$$
$$p_4 + r_4 = 9 + 10 = 19$$
$$p_5 + r_3 = 10 + 8 = 18$$
$$p_6 + r_2 = 17 + 5 = 22$$
$$p_7 + r_1 = 17 + 1 = 18$$
$$p_8 + r_0 = 20 + 0 = 20$$

$r_8 = \max_{1 \le i \le 8}(p_i + r_{8-i}) = 22$, achieved by a first piece of length $s_8 = 2$. Only 8 look-ups into the table.

Bottom-up rod cutting in code — and the cut itself

Added – tested against brute force over all 2^{n-1} cuts:

```
// Bottom-up rod cutting. p[i] = price of length i.
// r[j] = best revenue for length j,
// s[j] = first piece of an optimal cut of j.
vector<int> rod_cut(const vector<int>& p, int n,
                    vector<int>& s) {
    vector<int> r(n + 1, 0);       // r[0] = 0
    s.assign(n + 1, 0);
    for (int j = 1; j <= n; ++j) { // small rods first
        int best = INT_MIN;
        for (int i = 1; i <= j; ++i)   // first piece i
            if (p[i] + r[j - i] > best) {
                best = p[i] + r[j - i];
                s[j] = i;
            }
        r[j] = best;
    }
    return r;
}
void print_cut(const vector<int>& s, int n) {
    while (n > 0) { printf("%d ", s[n]); n -= s[n]; }
    printf("\n");
}
```

Reconstructing the solution. s_j is the first piece of an optimal cut of j; the rest is an optimal cut of $j - s_j$.

$n = 8$: $s_8 = 2$, then $8 - 2 = 6$, $s_6 = 6$, then 0.
Pieces $2 + 6$: revenue
$p_2 + p_6 = 5 + 17 = 22 = r_8$. ✓

Pattern

Store the *value* r_j and the *choice* s_j. Walk the choices to output the solution in $O(n)$ extra time.

The recipe

A DP needs a collection of subproblems with:

1. only polynomially many of them;
2. the original answer is easily computed from them;
3. a natural order "smallest first" with an easy recurrence from smaller ones.

Designing a DP — five steps

1. Define the subproblem (what does a table entry mean?)
2. Recurrence: the last/first decision
3. Base cases
4. Evaluation order
5. Reconstruct the solution from stored choices

Rod cutting: (1) r_j, best revenue for length j; (2) first piece i; (3) $r_0 = 0$; (4) $j = 1, 2, \ldots, n$; (5) follow s_j.

Edit distance

How far apart are two strings?

Three single-character edits, each costing 1:

- **substitute**: `shot` $\rightarrow$ `spot`
- **insert**: `ago` $\rightarrow$ `agog`
- **delete**: `hour` $\rightarrow$ `our`

The edit distance of x and y = the minimum number of edits that turn x into y.

Spell checkers, diff, DNA/protein comparison.

How far apart are two strings?

Three single-character edits, each costing 1:

- **substitute**: `shot` → `spot`
- **insert**: `ago` → `agog`
- **delete**: `hour` → `our`

The edit distance of x and y = the minimum number of edits that turn x into y.

Spell checkers, diff, DNA/protein comparison.

Same thing, drawn as an alignment (K&T):

```
o-currance
occurrence
```

- gap in x = insert, gap in y = delete
- mismatched column (a/e) = substitute

Cost = 1 gap + 1 mismatch = 2.

How far apart are two strings?

Three single-character edits, each costing 1:

- **substitute**: `shot` → `spot`
- **insert**: `ago` → `agog`
- **delete**: `hour` → `our`

The edit distance of x and y = the minimum number of edits that turn x into y.

Spell checkers, diff, DNA/protein comparison.

Same thing, drawn as an alignment (K&T):

```
o-currance
occurrence
```

- gap in x = insert, gap in y = delete
- mismatched column (a/e) = substitute

Cost = 1 gap + 1 mismatch = 2.

Hard-looking: *where* to insert and delete? Skiena's hint: ask what happens to the last characters. Skiena §8.2; Kleinberg & Tardos §6.6.

The last column of an optimal alignment

Subproblem: $D[i][j]$ = edit distance between the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$.

The last column of an optimal alignment

Subproblem: $D[i][j]$ = edit distance between the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$.

The last column of an optimal alignment of these prefixes is one of (K&T 6.15):

1. x_i over y_j: *match* (0) or *substitute* (1);
2. x_i over a gap: *delete* x_i (1);
3. a gap over y_j: *insert* y_j (1).

The last column of an optimal alignment

Subproblem: $D[i][j]$ = edit distance between the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$.

The last column of an optimal alignment of these prefixes is one of (K&T 6.15):

1. x_i over y_j: *match* (0) or *substitute* (1);
2. x_i over a gap: *delete* x_i (1);
3. a gap over y_j: *insert* y_j (1).

Remove that column: what is left is an optimal alignment of shorter prefixes.

The last column of an optimal alignment

Subproblem: $D[i][j]$ = edit distance between the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$.

The last column of an optimal alignment of these prefixes is one of (K&T 6.15):

1. x_i over y_j: *match* (0) or *substitute* (1);
2. x_i over a gap: *delete* x_i (1);
3. a gap over y_j: *insert* y_j (1).

Remove that column: what is left is an optimal alignment of shorter prefixes.

Recurrence

$$D[i][j] = \min \begin{cases} D[i-1][j-1] + [x_i \neq y_j] \\ D[i-1][j] + 1 \\ D[i][j-1] + 1 \end{cases}$$

Base: $D[i][0] = i,\ D[0][j] = j$.

The last column of an optimal alignment

Subproblem: $D[i][j]$ = edit distance between the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$.

The last column of an optimal alignment of these prefixes is one of (K&T 6.15):

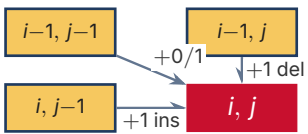
1. x_i over y_j: *match* (0) or *substitute* (1);
2. x_i over a gap: *delete* x_i (1);
3. a gap over y_j: *insert* y_j (1).

Remove that column: what is left is an optimal alignment of shorter prefixes.

Recurrence

$$D[i][j] = \min \begin{cases} D[i-1][j-1] + [x_i \neq y_j] \\ D[i-1][j] + 1 \\ D[i][j-1] + 1 \end{cases}$$

Base: $D[i][0] = i,\ D[0][j] = j$.



Fill row by row.

Filling the table: `shalt` → `should`

$y =$ `should` $\rightarrow j$

$x =$ `shalt` $\rightarrow i$	ε	**s**	**h**	**o**	**u**	**l**	**d**
ε	0	1	2	3	4	5	6
s	1						
h	2						
a	3						
l	4						
t	5						

$D[i][0] = i$ (delete i characters)
$D[0][j] = j$ (insert j characters)

Row 0 and column 0: turning a prefix into ε (or back) costs its length. Fill the rest row by row; each cell needs its left, upper and upper-left neighbours.

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	**0**					
h	2						
a	3						
l	4						
t	5						

Cell $(1,1)$: s $\rightarrow$ s

match s/s	$D[0][0] + 0 = 0$
delete s	$D[0][1] + 1 = 2$
insert s	$D[1][0] + 1 = 2$

$D[1][1] = \min = 0$

s $=$ s: the diagonal step is a free match.

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	**1**				
h	2						
a	3						
l	4						
t	5						

Cell $(1, 2)$: s $\rightarrow$ sh

substitute s/h	$D[0][1] + 1 = 2$
delete s	$D[0][2] + 1 = 3$
insert h	$D[1][1] + 1 = 1$

$D[1][2] = \min = 1$

Ties are fine: any minimiser gives an optimal answer.

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	**2**			
h	2						
a	3						
l	4						
t	5						

Cell $(1, 3)$: s $\rightarrow$ sho

substitute s/o	$D[0][2] + 1 = 3$
delete s	$D[0][3] + 1 = 4$
insert o	$D[1][2] + 1 = 2$

$D[1][3] = \min = 2$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	**3**		
h	2						
a	3						
l	4						
t	5						

Cell $(1, 4)$: s $\rightarrow$ shou

substitute s/u	$D[0][3] + 1 = 4$
delete s	$D[0][4] + 1 = 5$
insert u	$D[1][3] + 1 = 3$

$D[1][4] = \min = 3$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	**4**	
h	2						
a	3						
l	4						
t	5						

Cell $(1, 5)$: s $\rightarrow$ shoul

substitute s/l	$D[0][4] + 1 = 5$
delete s	$D[0][5] + 1 = 6$
insert l	$D[1][4] + 1 = 4$

$D[1][5] = \min = 4$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	**5**
h	2						
a	3						
l	4						
t	5						

Cell $(1, 6)$: s $\rightarrow$ should

substitute s/d	$D[0][5] + 1 = 6$
delete s	$D[0][6] + 1 = 7$
insert d	$D[1][5] + 1 = 5$

$D[1][6] = \min = 5$

Filling the table: shalt → should

$y = \texttt{should} \to j$

$x = \texttt{shalt} \to i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	**1**					
a	3						
l	4						
t	5						

Cell $(2, 1)$: sh → s

substitute h/s	$D[1][0] + 1 = 2$
delete h	$D[1][1] + 1 = 1$
insert s	$D[2][0] + 1 = 3$

$D[2][1] = \min = 1$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	**0**				
a	3						
l	4						
t	5						

Cell $(2, 2)$: sh $\rightarrow$ sh

match h/h	$D[1][1] + 0 = 0$
delete h	$D[1][2] + 1 = 2$
insert h	$D[2][1] + 1 = 2$

$D[2][2] = \min = 0$

sh $\rightarrow$ sh: two free matches, cost 0.

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	**1**			
a	3						
l	4						
t	5						

Cell $(2, 3)$: sh → sho

substitute h/o	$D[1][2] + 1 = 2$
delete h	$D[1][3] + 1 = 3$
insert o	$D[2][2] + 1 = 1$

$D[2][3] = \min = 1$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	**2**		
a	3						
l	4						
t	5						

Cell $(2, 4)$: sh $\rightarrow$ shou

substitute h/u $D[1][3] + 1 = 3$

delete h $D[1][4] + 1 = 4$

insert u $D[2][3] + 1 = 2$

$D[2][4] = \min = 2$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	**3**	
a	3						
l	4						
t	5						

Cell $(2, 5)$: sh → shoul

substitute h/l	$D[1][4] + 1 = 4$
delete h	$D[1][5] + 1 = 5$
insert l	$D[2][4] + 1 = 3$

$D[2][5] = \min = 3$

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	**4**
a	3						
l	4						
t	5						

Cell $(2, 6)$: sh $\rightarrow$ should

substitute h/d	$D[1][5] + 1 = 5$
delete h	$D[1][6] + 1 = 6$
insert d	$D[2][5] + 1 = 4$

$D[2][6] = \min = 4$

Filling the table: shalt → should

$y =$ should $\to j$

$x =$ shalt $\to i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	**2**	**1**	**1**	**2**	**3**	**4**
l	4						
t	5						

Row 3 (a) filled the same way, left to right.

Filling the table: shalt → should

$y =$ should $\rightarrow j$

$x =$ shalt $\rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	2	1	1	2	3	4
l	4	**3**	**2**	**2**	**2**	**2**	**3**
t	5						

Row 4 (l) filled the same way, left to right.

Filling the table: `shalt` → `should`

$y =$ `should` $\to j$

$x =$ `shalt` $\to i$	ε	**s**	**h**	**o**	**u**	**l**	**d**
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	2	1	1	2	3	4
l	4	3	2	2	2	2	3
t	5	**4**	**3**	**3**	**3**	**3**	**3**

Done: edit distance(`shalt`, `should`) $= D[5][6] = 3$. $(m+1)(n+1)$ cells, $O(1)$ work each $\Rightarrow O(mn)$ time.

Traceback: recovering the edits

$x = \texttt{shalt} \to i$ \ $y = \texttt{should} \to j$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	2	1	1	2	3	4
l	4	3	2	2	2	2	3
t	5	4	3	3	3	3	**3**

x

y

op

Start at the goal cell $(5, 6)$, value 3. Ask: which neighbour's value + its cost gives this value?

Traceback: recovering the edits

$y = \texttt{should} \rightarrow j$

$x = \texttt{shalt} \rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	2	1	1	2	3	4
l	4	3	2	2	2	**2**	3
t	5	4	3	3	3	3	**3**

x	t
y	d
op	**s**

substitute t/d: $D[5][6] = D[4][5] + 1$

Traceback: recovering the edits

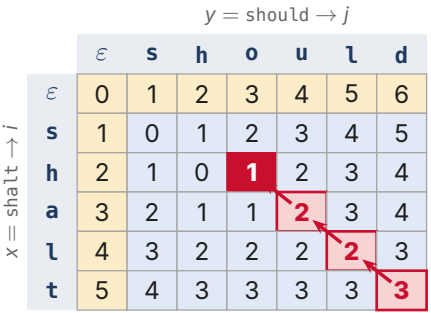
$y = \texttt{should} \rightarrow j$

$x = \texttt{shalt} \rightarrow i$	ε	s	h	o	u	l	d
ε	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
h	2	1	0	1	2	3	4
a	3	2	1	1	**2**	3	4
l	4	3	2	2	2	**2**	3
t	5	4	3	3	3	3	**3**

x	l	t
y	l	d
op	**M**	**S**

match l/l: $D[4][5] = D[3][4] + 0$

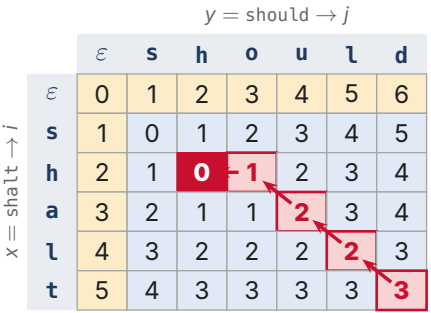
Traceback: recovering the edits



x	a	l	t
y	u	l	d
op	**S**	**M**	**S**

substitute a/u: $D[3][4] = D[2][3] + 1$

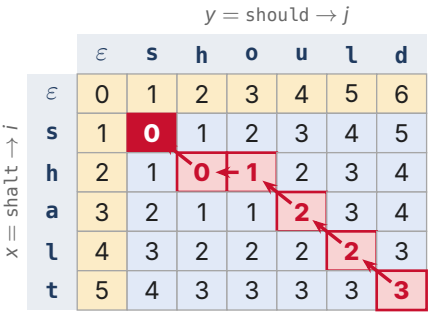
Traceback: recovering the edits



x	–	a	l	t
y	o	u	l	d
op	**I**	**S**	**M**	**S**

insert o: $D[2][3] = D[2][2] + 1$

Traceback: recovering the edits



x	h	–	a	l	t
y	h	o	u	l	d
op	**M**	**I**	**S**	**M**	**S**

match h/h: $D[2][2] = D[1][1] + 0$

Traceback: recovering the edits

$y = \texttt{should} \rightarrow j$

$x = \texttt{shalt} \downarrow i$	ε	s	h	o	u	l	d
ε	**0**	1	2	3	4	5	6
s	1	**0**	1	2	3	4	5
h	2	1	**0**	**1**	2	3	4
a	3	2	1	1	**2**	3	4
l	4	3	2	2	2	**2**	3
t	5	4	3	3	3	3	**3**

x	s	h	–	a	l	t
y	s	h	o	u	l	d
op	**M**	**M**	**I**	**S**	**M**	**S**

match s/s: $D[1][1] = D[0][0] + 0$.

Reached $(0, 0)$. The alignment is read *backwards*; cost $= 3$ non-match columns.

General costs and the grid-graph view

Gap cost δ, mismatch cost α_{pq} — the same recurrence:

$$\mathrm{OPT}(i,j) = \min \begin{cases} \alpha_{x_i y_j} + \mathrm{OPT}(i-1, j-1) \\ \delta + \mathrm{OPT}(i-1, j) \\ \delta + \mathrm{OPT}(i, j-1) \end{cases}$$

General costs and the grid-graph view

Gap cost δ, mismatch cost α_{pq} — the same recurrence:

$$\mathrm{OPT}(i,j) = \min \begin{cases} \alpha_{x_i y_j} + \mathrm{OPT}(i-1, j-1) \\ \delta + \mathrm{OPT}(i-1, j) \\ \delta + \mathrm{OPT}(i, j-1) \end{cases}$$

Alignment = shortest path

Grid DAG on cells (i,j): right/down edges cost δ, diagonal into (i,j) costs $\alpha_{x_i y_j}$. $\mathrm{OPT}(i,j)$ = shortest $(0,0) \to (i,j)$ path.

General costs and the grid-graph view

Gap cost δ, mismatch cost α_{pq} — the same recurrence:

$$\mathrm{OPT}(i,j) = \min \begin{cases} \alpha_{x_i y_j} + \mathrm{OPT}(i-1, j-1) \\ \delta + \mathrm{OPT}(i-1, j) \\ \delta + \mathrm{OPT}(i, j-1) \end{cases}$$

Alignment = shortest path

Grid DAG on cells (i,j): right/down edges cost δ, diagonal into (i,j) costs $\alpha_{x_i y_j}$. $\mathrm{OPT}(i,j)$ = shortest $(0,0) \to (i,j)$ path.

Example: `mean` vs `name`, $\delta = 2$; vowel/vowel or consonant/consonant mismatch 1, vowel/consonant 3. Cost 6.

$y = \texttt{name} \to j$

$x = \texttt{mean} \to i$	ε	n	a	m	e
ε	**0**	2	4	6	8
m	2	**1**	3	4	6
e	4	**3**	2	4	4
a	6	5	**3**	5	5
n	8	6	5	**4**	**6**

x	m	e	a	n	–
y	n	–	a	m	e

Edit distance in code; time and space

Added – tested against plain recursion on random strings:

```cpp
// D[i][j] = edit distance of x[0..i) and y[0..j)
int edit_distance(const string& x, const string& y) {
    int m = x.size(), n = y.size();
    vector<vector<int>> D(m + 1, vector<int>(n + 1));
    for (int i = 0; i <= m; ++i) D[i][0] = i; // deletes
    for (int j = 0; j <= n; ++j) D[0][j] = j; // inserts
    for (int i = 1; i <= m; ++i)
        for (int j = 1; j <= n; ++j) {
            int sub = (x[i - 1] == y[j - 1]) ? 0 : 1;
            D[i][j] = min({ D[i-1][j-1] + sub,  // match/sub
                            D[i-1][j] + 1,      // delete
                            D[i][j-1] + 1 });   // insert
        }
    return D[m][n];
}
```

Cost

Time $O(mn)$, space $O(mn)$.

- Plain recursion branches 3 ways: $\Omega(3^{\min(m,n)})$ calls.
- Value only: keep two rows $\Rightarrow O(\min(m,n))$ space.
- Alignment in linear space too: Hirschberg's divide and conquer.

Small changes, new problems (Skiena §8.2.4)

Problem	What changes	Answer
Weighted edits	substitution / indel costs depend on the characters (keyboard distance, DNA scores)	$D[m][n]$
Substring matching (find x inside text y)	$D[0][j] = 0$: a match may *start* anywhere in y	$\min_j D[m][j]$
Longest common subsequence	forbid substitution (cost > 2)	$\mathrm{LCS} = \frac{m+n-D[m][n]}{2}$

Same table, same order, same traceback — only initialisation, costs and goal cell move.

Wrap-up

Summary

	Rod cutting	**Edit distance**
Subproblem	r_j: best revenue, rod of length j	$D[i][j]$: distance of prefixes $x_{1..i}$, $y_{1..j}$
Decision	length of the *first* piece (j choices)	content of the *last* column (3 choices)
Base	$r_0 = 0$	$D[i][0] = i$, $D[0][j] = j$
Order	$j = 1, \ldots, n$	row by row
Time / space	$\Theta(n^2)$ / $\Theta(n)$	$\Theta(mn)$ / $\Theta(mn)$ (value: $O(\min(m, n))$)
Solution	follow s_j	trace back from (m, n)

Time of a DP = (number of subproblems) $\times$ (choices per subproblem).

Check yourself

Quiz

1. Every cut now costs a fixed fee c. Change the rod-cutting recurrence.
2. Show that `algorithm` and `rhythm` have edit distance 6 (Goodrich & Tamassia, C-12.9).
3. With gap cost δ and mismatch cost α_{ae}, when is `o–currance/occurrence` cheaper than `o–curr–ance/occurre–nce`?

Check yourself

Quiz

1. Every cut now costs a fixed fee c. Change the rod-cutting recurrence.
2. Show that `algorithm` and `rhythm` have edit distance 6 (Goodrich & Tamassia, C-12.9).
3. With gap cost δ and mismatch cost α_{ae}, when is `o–currance/occurrence` cheaper than `o–curr–ance/occurre–nce`?

Answers. (1) $r_n = \max\big(p_n,\ \max_{1 \le i < n}(p_i + r_{n-i} - c)\big)$: selling whole needs no cut. (2) $D[9][6] = 6$, e.g. delete `a`, `l`, `g`; substitute `o`→`r`, `r`→`h`, `i`→`y`; match `thm`. The table confirms nothing cheaper exists. (3) Iff $\delta + \alpha_{ae} < 3\delta$, i.e. $\alpha_{ae} < 2\delta$.